

Fixed-point combinator

In combinatory logic for computer science, a **fixed-point combinator** (or **fixpoint combinator**)^{[1]:p.26} is a higher-order function (i.e., a function which takes a function as argument) that returns some fixed point (a value that is mapped to itself) of its argument function, if one exists.

Formally, if **fix** is a fixed-point combinator and the function *f* has one or more fixed points, then **fix** *f* is one of these fixed points, i.e.,

$$\mathbf{fix} \ f = f \ (\mathbf{fix} \ f).$$

Fixed-point combinators can be defined in the lambda calculus and in functional programming languages, and provide a means to allow for recursive definitions.

Y combinator in lambda calculus

In the classical untyped lambda calculus, every function has a fixed point. A particular implementation of **fix** is Haskell Curry's paradoxical combinator *Y*, given by^{[2]:131}^[note 1]^[note 2]

$$Y = \lambda f. (\lambda x. f \ (x \ x)) \ (\lambda x. f \ (x \ x))$$

(Here using the standard notations and conventions of lambda calculus: *Y* is a function that takes one argument *f* and returns the entire expression following the first period; the expression $\lambda x. f \ (x \ x)$ denotes a function that takes one argument *x*, thought of as a function, and returns the expression $f \ (x \ x)$, where $(x \ x)$ denotes *x* applied to itself. Juxtaposition of expressions denotes function application, is left-associative, and has higher precedence than the period.)

Verification

The following calculation verifies that **Y***g* is indeed a fixed point of the function *g*:

$$\begin{aligned} Y \ g &\triangleq (\lambda f. (\lambda x. f \ (x \ x)) \ (\lambda x. f \ (x \ x))) \ g && \text{by the definition of } Y \\ &=_{\beta} (\lambda x. g \ (x \ x)) \ (\lambda x. g \ (x \ x)) && \text{by } \beta\text{-reduction: replacing the formal argument } f \text{ of } Y \text{ with the actual argument } g \\ &=_{\beta} g \ ((\lambda x. g \ (x \ x)) \ (\lambda x. g \ (x \ x))) && \text{by } \beta\text{-reduction: replacing the formal argument } x \text{ of the first function with the actual argument } (\lambda x. g \ (x \ x)) \\ &\equiv g \ (Y \ g) && \text{by second equality, above} \end{aligned}$$

The lambda term *g* (**Y** *g*) may not, in general, β-reduce to the term **Y** *g*. However, both terms β-reduce to the same term, as shown.

Uses

Applied to a function with one variable, the *Y* combinator usually does not terminate. More interesting results are obtained by applying the *Y* combinator to functions of two or more variables. The added variables may be used as a counter, or index. The resulting function behaves like a *while* or a *for* loop in an imperative language.

Used in this way, the *Y* combinator implements simple recursion. The lambda calculus does not allow a function to appear as a term in its own definition as is possible in many programming languages, but a function can be passed as an argument to a higher-order function that applies it in a recursive manner.

The *Y* combinator may also be used in implementing Curry's paradox. The heart of Curry's paradox is that untyped lambda calculus is unsound as a deductive system, and the *Y* combinator demonstrates this by allowing an anonymous expression to represent zero, or even many values. This is inconsistent in mathematical logic.

Example implementations

An example implementation of *Y* in the language R is presented below:

```
Y <- \(f) {  
  g <- \(x) f(x(x))  
  g(g)  
}
```

This can then be used to implement factorial as follows:

```
fact <- \(f) \(n)  
  if (n == 0) 1 else n * f(n - 1)  
Y(fact)(5) # yields 5! = 120
```

Y is only needed when function names are absent. Substituting all the definitions into one line so that function names are not required gives:

```
(\(f) (\(x) f(x(x)))(\(x) f(x(x))) (\(f) \(n) if (n == 0) 1 else n * f(n - 1)) (5)
```

This works because R uses lazy evaluation.

Languages that use strict evaluation, such as Python, C++, and other strict programming languages, can often express *Y*; however, any implementation is useless in practice since it loops indefinitely until terminating via a stack overflow.

Fixed-point combinator

The *Y* combinator is an implementation of a fixed-point combinator in lambda calculus. Fixed-point combinators may also be easily defined in other functional and imperative languages. The implementation in lambda calculus is more difficult due to limitations in lambda calculus. The fixed-point combinator may be used in a number of different areas:

- General mathematics
- Untyped lambda calculus
- Typed lambda calculus
- Functional programming
- Imperative programming

Fixed-point combinators may be applied to a range of different functions, but normally will not terminate unless there is an extra parameter. When the function to be fixed refers to its parameter, another call to the function is invoked, so the calculation never gets started. Instead, the extra parameter is used to trigger the start of the calculation.

The type of the fixed point is the return type of the function being fixed. This may be a real or a function or any other type.

In the untyped lambda calculus, the function to apply the fixed-point combinator to may be expressed using an encoding, like Church encoding. In this case particular lambda terms (which define functions) are considered as values. "Running" (beta reducing) the fixed-point combinator on the encoding gives a lambda term for the result which may then be interpreted as fixed-point value.

Alternately, a function may be considered as a lambda term defined purely in lambda calculus.

These different approaches affect how a mathematician and a programmer may regard a fixed-point combinator. A mathematician may see the Y combinator applied to a function as being an expression satisfying the fixed-point equation, and therefore a solution.

In contrast, a person only wanting to apply a fixed-point combinator to some general programming task may see it only as a means of implementing recursion.

Values and domains

Many functions do not have any fixed points, for instance $f : \mathbb{N} \rightarrow \mathbb{N}$ with $f(n) = n + 1$. Using Church encoding, natural numbers can be represented in lambda calculus, and this function f can be defined in lambda calculus. However, its domain will now contain *all* lambda expressions, not just those representing natural numbers. The Y combinator, applied to f , will yield a fixed-point for f , but this fixed-point won't represent a natural number. If trying to compute $Y f$ in an actual programming language, an infinite loop will occur.

Function versus implementation

The fixed-point combinator may be defined in mathematics and then implemented in other languages. General mathematics defines a function based on its extensional properties.^[3] That is, two functions are equal if they perform the same mapping. Lambda calculus and programming languages regard function identity as an intensional property. A function's identity is based on its implementation.

A lambda calculus function (or term) is an implementation of a mathematical function. In the lambda calculus there are a number of combinators (implementations) that satisfy the mathematical definition of a fixed-point combinator.

Definition of the term "combinator"

Combinatory logic is a higher-order functions theory. A combinator is a *closed* lambda expression, meaning that it has no free variables. The combinators may be combined to direct values to their correct places in the expression without ever naming them as variables.

Recursive definitions and fixed-point combinators

Fixed-point combinators can be used to implement recursive definition of functions. However, they are rarely used in practical programming.^[4] Strongly normalizing type systems such as the simply typed lambda calculus disallow non-termination and hence fixed-point combinators often cannot be assigned a type or require complex type system features. Furthermore, fixed-point combinators are often inefficient compared to other strategies for implementing recursion, as they require more function reductions and construct and take apart a tuple for each group of mutually recursive definitions.^[1]:page 232

The factorial function

The factorial function provides a good example of how a fixed-point combinator may be used to define recursive functions. The standard recursive definition of the factorial function in mathematics can be written as

$$\mathbf{fact} \ n = \begin{cases} 1 & \text{if } n = 0 \\ n \times \mathbf{fact}(n - 1) & \text{otherwise.} \end{cases}$$

where n is a non-negative integer. Implementing this in lambda calculus, where integers are represented using Church encoding, encounters the problem that the lambda calculus disallows the name of a function ('fact') to be used in the function's definition. This can be circumvented using a fixed-point combinator **fix** as follows.

Define a function F of two arguments f and n :

$$F \ f \ n = (\mathbf{IsZero} \ n) \ 1 \ (\mathbf{multiply} \ n \ (f \ (\mathbf{pred} \ n)))$$

(Here $(\mathbf{IsZero} \ n)$ is a function that takes two arguments and returns its first argument if $n=0$, and its second argument otherwise; $\mathbf{pred} \ n$ evaluates to $n-1$.)

Now define $\mathbf{fact} = \mathbf{fix} \ F$. Then $\mathbf{fact}$ is a fixed-point of F , which gives

$$\begin{aligned} \mathbf{fact} \ n &= F \ \mathbf{fact} \ n \\ &= (\mathbf{IsZero} \ n) \ 1 \ (\mathbf{multiply} \ n \ (\mathbf{fact} \ (\mathbf{pred} \ n))) \end{aligned}$$

as desired.

Fixed-point combinators in lambda calculus

The Y combinator, discovered by Haskell Curry, is defined as

$$Y = \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$$

Other fixed-point combinators

In untyped lambda calculus fixed-point combinators are not especially rare. In fact there are infinitely many of them.^[5] In 2005 Mayer Goldberg showed that the set of fixed-point combinators of untyped lambda calculus is recursively enumerable.^[6]

The Y combinator can be expressed in the SKI-calculus as

$$Y = S(K(SII))(S(S(KS)K)(K(SII))) = SS(S(S(KS)K))(K(SII))$$

Additional combinators (B, C, K, W system) allow for much shorter encodings. With **U** = **SII** the self-application combinator, since **S(Kx)yz = x(yz) = Bxyz** and **Sx(Ky)z = xzy = Cxyz**, the above becomes

$$Y = S(KU)(SB(KU)) = BU(CBU) \ ; \ Y = SSI(BWB)$$

The shortest fixed-point combinator in the SK-calculus using S and K combinators only, found by John Tromp, is

$$Y' = SSK(S(K(SS(SSSK))))K = WC(SB(C(WC)))$$

although note that it is not in normal form, which is longer. This combinator corresponds to the lambda expression

$$Y' = (\lambda xy. xyx)(\lambda yx. y(xyx))$$

The following fixed-point combinator is simpler than the Y combinator, and β -reduces into the Y combinator; it is sometimes cited as the Y combinator itself:

$$X = \lambda f. (\lambda x. xx)(\lambda x. f(xx)) \ ; \ Xf = U(BfU)$$

Another common fixed-point combinator is the Turing fixed-point combinator (named after its discoverer, Alan Turing):^{[7][2]:132}

$$\Theta = (\lambda xy. y(xxy)) (\lambda xy. y(xxy)) = SII(S(K(SI))(SII)) = U(B(SI)U)$$

Its advantage over **Y** is that **Θ f** beta-reduces to **f (Θ f)**,^[note 3] whereas **Y f** and **f (Y f)** only beta-reduce to a common term.

Θ also has a simple call-by-value form:

$$\Theta_v = (\lambda xy. y(\lambda z. xxyz)) (\lambda xy. y(\lambda z. xxyz))$$

The analog for mutual recursion is a *polyvariadic fix-point combinator*,^{[8][9][10]} which may be denoted **Y***.

Strict fixed-point combinator

In a strict programming language the Y combinator will expand until stack overflow, or never halt in case of tail call optimization.^[11] The Z combinator will work in strict languages (also called eager languages, where applicative evaluation order is applied). The Z combinator has the next argument defined explicitly, preventing the expansion of Zg in the right-hand side of the definition:^[12]

$$Zgv = g(Zg)v .$$

and in lambda calculus it is an eta-expansion of the Y combinator:

$$Z = \lambda f. (\lambda x. f(\lambda v. xxv)) (\lambda x. f(\lambda v. xxv)) .$$

Non-standard fixed-point combinators

If F is a fixed-point combinator in untyped lambda calculus, then there is:

$$F = \lambda x. Fx = \lambda x. x(Fx) = \lambda x. x(x(Fx)) = \dots$$

Terms that have the same Böhm tree as a fixed-point combinator, i.e., have the same infinite extension $\lambda x. x(x(x \dots))$, are called *non-standard fixed-point combinators*. Any fixed-point combinator is also a non-standard one, but not all non-standard fixed-point combinators are fixed-point combinators because some of them fail to satisfy the fixed-point equation that defines the "standard" ones. These combinators are called *strictly non-standard fixed-point combinators*; an example is the following combinator:

$$N = BU(BU)B$$

where

$$\begin{aligned} B &= \lambda xyz. x(yz) \\ U &= \lambda x. xx \end{aligned}$$

since

$$N = \lambda x. Nx = \lambda x. x(N_2x) = \lambda x. x(x(N_3x)) = \lambda x. x(x(x(N_4x))) = \dots$$

where N_i are modifications of N created on the fly which add i instances of x at once into the chain while being replaced with N_{i+1} .

The set of non-standard fixed-point combinators is not recursively enumerable.^[6]

Implementation in other languages

The Y combinator is a particular implementation of a fixed-point combinator in lambda calculus. Its structure is determined by the limitations of lambda calculus. It is not necessary or helpful to use this structure in implementing the fixed-point combinator in other languages.

Simple examples of fixed-point combinators implemented in some programming paradigms are given below.

Lazy functional implementation

In a language that supports lazy evaluation, as in Haskell, it is possible to define a fixed-point combinator using the defining equation of the fixed-point combinator which is conventionally named `fix`. Since Haskell has lazy data types, this combinator can also be used to define fixed points of data constructors (and not only to implement recursive functions). The definition is given here, followed by some usage examples. In Hackage, the original sample is:^[13]

```
fix, fix' :: (a -> a) -> a
fix f = let x = f x in x           -- Lambda dropped. Sharing.
                                   -- Original definition in Data.Function.
-- alternative:
fix' f = f (fix' f)               -- Lambda lifted. Non-sharing.

fix (\x -> 9)                     -- this evaluates to 9

fix (\x -> 3:x)                   -- evaluates to the lazy infinite list [3,3,3,...]

fact = fix fac                   -- evaluates to the factorial function
  where fac f 0 = 1
        fac f x = x * f (x-1)

fact 5                           -- evaluates to 120
```

Strict functional implementation

In a strict functional language, as illustrated below with OCaml, the argument to f is expanded beforehand, yielding an infinite call sequence,

$$f(f \dots (f(\text{fix } f)) \dots) x.$$

This may be resolved by defining `fix` with an extra parameter.

```
let rec fix f x = f (fix f) x (* note the extra x; hence fix f = \x-> f (fix f) x *)

let factabs fact = function (* factabs has extra level of lambda abstraction *)
  0 -> 1
  | x -> x * fact (x-1)

let _ = (fix factabs) 5        (* evaluates to "120" *)
```

In a multi-paradigm functional language (one decorated with imperative features), such as Lisp, Peter Landin suggested the use of a variable assignment to create a fixed-point combinator,^[14] as in the below example using Scheme:

```
(define Y!
  (lambda (f)
    ((lambda (g)
      (set! g (f (lambda (x) (g x)))))) ;; (set! g expr) assigns g the value of expr,
      g)                                ;; replacing g's initial value of #f, creating
    #f))                                ;; thus the truly self-referential value in g
```

Using a lambda calculus with axioms for assignment statements, it can be shown that $Y!$ satisfies the same fixed-point law as the call-by-value Y combinator:^{[15][16]}

$$(Y! \lambda x. e) e' = (\lambda x. e) (Y! \lambda x. e) e'$$

In more idiomatic modern Scheme usage, this would typically be handled via a `letrec` expression, as lexical scope was introduced to Lisp in the 1970s:

```
(define Y*  
  (lambda (f)  
    (letrec  
      ((g  
        (f (lambda (x) (g x)))))  
      (g)))  
    ;; (letrec ((g expr)) ...) locally defines g  
    ;; as expr recursively: g in expr refers to  
    ;; that same g being defined, g = f (λx. g x)  
    ;; ((Y* f) ...) = (g ...) = ((f (λx. g x)) ...)
```

Or without the internal label:

```
(define Y*  
  (lambda (f)  
    ((lambda (i) (i i))  
     (lambda (i)  
       (f (lambda x  
            (apply (i i) x)))))))
```

Imperative language implementation

This example is a slightly interpretive implementation of a fixed-point combinator. A class is used to contain the *fix* function, called *fixer*. The function to be fixed is contained in a class that inherits from *fixer*. The *fix* function accesses the function to be fixed as a virtual function. As for the strict functional definition, *fix* is explicitly given an extra parameter *x*, which means that lazy evaluation is not needed.

```
template <typename R, typename D>  
class fixer  
{  
public:  
    R fix(D x)  
    {  
        return f(x);  
    }  
private:  
    virtual R f(D) = 0;  
};  
  
class fact : public fixer<long, long>  
{  
    virtual long f(long x)  
    {  
        if (x == 0)  
        {  
            return 1;  
        }  
        return x * fix(x-1);  
    }  
};  
  
long result = fact().fix(5);
```

Another example can be shown to demonstrate SKI combinator calculus (with given bird name from combinatory logic) being used to build up Z combinator to achieve tail call-like behavior through trampolining:

```
var K = a => b => a; // Kestrel  
var S = a => b => c => a(c)(b(c)); // Starling  
var I = S(K)(K); // Identity  
var B = S(K(S))(K); // Bluebird
```


[illegible]

Typing

In System F (polymorphic lambda calculus) a polymorphic fixed-point combinator has type^[17]

$$\forall a.(a \rightarrow a) \rightarrow a$$

where $\mathbf{a}$ is a type variable. That is, if the type of $\mathbf{fix}\ f$ fulfilling the equation $\mathbf{fix}\ f = f(\mathbf{fix}\ f)$ is $\mathbf{a}$, – the most general type, – then the type of $\mathbf{f}$ is $\mathbf{a} \rightarrow \mathbf{a}$. So then, $\mathbf{fix}$ takes a function which maps $\mathbf{a}$ to $\mathbf{a}$ and uses it to return a value of type $\mathbf{a}$.

In the simply typed lambda calculus extended with recursive data types, fixed-point operators can be written, but the type of a "useful" fixed-point operator (one whose application always returns) may be restricted.

In the simply typed lambda calculus, the fixed-point combinator Y cannot be assigned a type^[18] because at some point it would deal with the self-application sub-term $\mathbf{x} \mathbf{x}$ by the application rule:

$$\frac{\Gamma \vdash x:t_1 \rightarrow t_2 \quad \Gamma \vdash x:t_1}{\Gamma \vdash x \ x:t_2}$$

where $\mathbf{x}$ has the infinite type $\mathbf{t}_1 = \mathbf{t}_1 \rightarrow \mathbf{t}_2$. No fixed-point combinator can in fact be typed; in those systems, any support for recursion must be explicitly added to the language.

Type for the Y combinator

In programming languages that support named recursive data types, the unbounded recursion in $t := t \rightarrow a$, which creates the would-be infinite type t , is broken by naming the type t explicitly, as e.g. type $R\ a$ which is defined so as to be isomorphic to (or just to be a synonym of) the type $R\ a \rightarrow a$. Thus $R\ a := R\ a \rightarrow a$ is recognized as a recursive type. A datum (value) of the type $R\ a$ is created by simply tagging a value that is a function that has the type $R\ a \rightarrow a$, by the data constructor tag for the type $R\ a$.

For example, in the following Haskell code, let `Rec` be that tag name, and thus `Rec` and `app` be the names of the two directions of the isomorphism, with types:^{[19][20]}

```
Rec :: (R a -> a) -> R a
app :: R a -> (R a -> a)
```

(where `::` means "has type"). This lets us write:

```
newtype R a = Rec { app :: R a -> a }
-- app (Rec g) = g      -- g :: R a -> a
--   Rec g      :: R a
-- app          :: R a -> (R a -> a)

y :: (a -> a) -> a
y f = (\ x -> f (app x x)) (Rec (\ x -> f (app x x)))
--   x :: R a
--   app x :: R a -> a
--   app x x :: a
```

The usual lambda-calculus definition $Y\ f = U(\lambda x. f\ (U\ x))$ (where $U\ x = x\ x$, and thus the term $U(\lambda x. f\ (U\ x))$ witnesses the equivalence $x\ x = f\ (x\ x)$) contains at its core the self-application $x\ x$, which is untypeable in the statically typed programming languages. But the expression `app x x` above is typeable. Equivalently, we can redefine the self-application combinator U and use it as

```
U' g = g (Rec g)
y f = U' (\ Rec g -> f (U' g))
--   g (Rec g) = f (g (Rec g))

U'' x = app x x
y f = U'' (Rec (\ x -> f (U'' x)))
--   app x x = f (app x x)
```

Or in OCaml:

```
type 'a recc = In of ('a recc -> 'a)
let out (In x) = x

let y f = (fun x a -> f (out x x) a) (In (fun x a -> f (out x x) a))
```

Alternatively:

```
let y f = (fun x -> f (fun z -> out x x z)) (In (fun x -> f (fun z -> out x x z)))
```

Some languages allow for explicitly marking general types as being recursive even without naming them. Doing that also allows for defining the Y combinator in such languages.

General information

Because fixed-point combinators can be used to implement recursion, it is possible to use them to describe specific types of recursive computations, such as those in [fixed-point iteration](#), [iterative methods](#), [recursive join in relational databases](#), [data-flow analysis](#), FIRST and FOLLOW sets of non-terminals in a [context-free grammar](#), [transitive closure](#), and other types of [closure](#) operations.

A function for which *every* input is a fixed point is called an [identity function](#). Formally:

$$\forall x(f\ x = x)$$

In contrast to [universal quantification](#) over all x , a fixed-point combinator constructs *one* value that is a fixed point of f . The remarkable property of a fixed-point combinator is that it constructs a fixed point for an *arbitrary given* function f .

Other functions have the special property that, after being applied once, further applications don't have any effect. More formally:

$$\forall x(f\ (f\ x) = f\ x)$$

Such functions are called [idempotent](#) (see also [Projection \(mathematics\)](#)). An example of such a function is the function that returns 0 for all even integers, and 1 for all odd integers.

In [lambda calculus](#), from a computational point of view, applying a fixed-point combinator to an identity function or an idempotent function typically results in non-terminating computation. For example, obtaining

$$(Y\ \lambda x.x) = (\lambda x. (\lambda x.x)(x\ x))\ (\lambda x. (\lambda x.x)(x\ x))$$

where the resulting term can only reduce to itself and represents an infinite loop.

Fixed-point combinators do not necessarily exist in more restrictive models of computation. For instance, they do not exist in [simply typed lambda calculus](#).

The Y combinator allows [recursion](#) to be defined as a set of [rewrite rules](#),^[21] without requiring native recursion support in the language.^[22]

In programming languages that support [anonymous functions](#), fixed-point combinators allow the definition and use of anonymous [recursive](#) functions, i.e., without having to [bind](#) such functions to [identifiers](#). In this setting, the use of fixed-point combinators is sometimes called [anonymous recursion](#).^{[note 4][23]}

See also

- [Anonymous function](#)
- [Fixed-point iteration](#)
- [Lambda calculus#Recursion and fixed points](#)

- Lambda lifting
- Lawvere's fixed-point theorem
- Let expression

Notes

1. Throughout this article, the syntax rules given in Lambda calculus § Notation are used, to save parentheses.
2. According to Barendregt p.132, the name originated from Curry.
3. $\Theta f \equiv (\lambda xy. y(xxy)) (\lambda xy. y(xxy)) f \rightarrow (\lambda y. y ((\lambda xy. y(xxy)) (\lambda xy. y(xxy)) y)) f \rightarrow f ((\lambda xy. y(xxy)) (\lambda xy. y(xxy)) f) \equiv f (\Theta f)$
4. This terminology appears to be largely [[mathematical folklore]], but it appears in:
 - Trey Nash, *Accelerated C# 2008*, Apress, 2007, ISBN 1-59059-873-3, p. 462—463. Derived substantially from Wes Dyer (<http://www.linkedin.com/pub/wes-dyer/2/727/a39>)'s blog (see next item).
 - Wes Dyer Anonymous Recursion in C# (<https://docs.microsoft.com/en-us/archive/blogs/wesdyer/anonymous-recursion-in-c>), February 02, 2007, contains a substantially similar example found in the book above, but accompanied by more discussion.

References

1. Peyton Jones, Simon L. (1987). *The Implementation of Functional Programming* (<https://www.microsoft.com/en-us/research/uploads/prod/1987/01/slpj-book-1987.pdf>) (PDF). Prentice Hall International.
2. Henk Barendregt (1985). *The Lambda Calculus – Its Syntax and Semantics*. Studies in Logic and the Foundations of Mathematics. Vol. 103. Amsterdam: North-Holland. ISBN 0444867481.
3. Selinger, Peter (2001–2013). "Lecture notes on the lambda calculus: Expository course notes" (<https://www.mscs.dal.ca/~selinger/papers/#lambdanotes>) (PDF). p. 6.
4. "For those of us who don't know what a Y-Combinator is or why it's useful, ..." (<https://news.ycombinator.com/item?id=17953711>) *Hacker News*. Retrieved 2 August 2020.
5. Bimbó, Katalin (27 July 2011). *Combinatory Logic: Pure, Applied and Typed* (<https://books.google.com/books?id=iQjMBQAAQBAJ&q=%22fixed-point+combinator%22>). CRC Press. p. 48. ISBN 9781439800010.
6. Goldberg, 2005
7. Alan Mathison Turing (December 1937). "The μ -function in λ - K -conversion". *Journal of Symbolic Logic*. **2** (4): 164. JSTOR 2268281 (<https://www.jstor.org/stable/2268281>).
8. "Many faces of the fixed-point combinator" (<http://okmij.org/ftp/Computation/fixed-point-combinators.html#Poly-variadic>). *okmij.org*.
9. Polyvariadic Y in pure Haskell98 (<http://osdir.com/ml/lang.haskell.cafe/2003-10/msg00211.html>) Archived (<https://web.archive.org/web/20160304101809/http://osdir.com/ml/lang.haskell.cafe/2003-10/msg00211.html>) 2016-03-04 at the Wayback Machine, lang.haskell.cafe, October 28, 2003
10. "recursion - Fixed-point combinator for mutually recursive functions?" (<https://stackoverflow.com/questions/4899113/fixed-point-combinator-for-mutually-recursive-functions>). *Stack Overflow*.

11. Bene, Adam (17 August 2017). "Fixed-Point Combinators in JavaScript" (<https://blog.benestudio.co/fixed-point-combinators-in-javascript-c214c15ff2f6>). *Bene Studio*. Medium. Retrieved 2 August 2020.
12. "CS 6110 S17 Lecture 5. Recursion and Fixed-Point Combinators" (<https://www.cs.cornell.edu/courses/cs6110/2017sp/lectures/lec05.pdf>) (PDF). *Cornell University*. 4.1 A CBV Fixed-Point Combinator.
13. The original definition in Data.Function (<https://hackage.haskell.org/package/base-4.10.0.0/docs/src/Data.Function.html#fix>).
14. Landin, P. J. (January 1964). "The mechanical evaluation of expressions". *The Computer Journal*. **6** (4): 308–320. doi:10.1093/comjnl/6.4.308 (<https://doi.org/10.1093%2Fcomjnl%2F6.4.308>).
15. Felleisen, Matthias (1987). *The Lambda-v-CS Calculus* (<https://www2.ccs.neu.edu/racket/pubs/#felleisen87>). Indiana University.
16. Talcott, Carolyn (1985). *The Essence of Rum: A theory of the intensional and extensional aspects of Lisp-type computation* (Ph.D. thesis). Stanford University.
17. Girard, Jean-Yves (1986). "The system F of variable types, fifteen years later". *Theoretical Computer Science*. **45** (2): 159–192. doi:10.1016/0304-3975(86)90044-7 (<https://doi.org/10.1016%2F0304-3975%2886%2990044-7>). MR 0867281 (<https://mathscinet.ams.org/mathscinet-getitem?mr=0867281>). See especially p. 180.
18. An Introduction to the Lambda Calculus (<http://cs.anu.edu.au/courses/COMP3610/lectures/Lambda.pdf>) Archived (<https://web.archive.org/web/20140408014716/http://cs.anu.edu.au/courses/COMP3610/lectures/Lambda.pdf>) 2014-04-08 at the Wayback Machine
19. Haskell mailing list thread on How to define Y combinator in Haskell (<https://groups.google.com/g/fa.haskell/c/8KYrbeFBbYs>), 15 September 2006
20. Geuvers, Herman; Verkoelen, Joep, *On Fixed point and Looping Combinators in Type Theory* (<https://www.cs.ru.nl/~herman/PUBS/TLCpaper.pdf>) (PDF), CiteSeerX 10.1.1.158.1478 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.158.1478>)
21. Friedman, Daniel P.; Felleisen, Matthias (1986). "Chapter 9 - Lambda The Ultimate". *The Little Lisper*. Science Research Associates. p. 179. "In the chapter we have derived a Y-combinator which allows us to write recursive functions of one argument without using define."
22. Vanier, Mike (14 August 2008). "The Y Combinator (Slight Return) or: How to Succeed at Recursion Without Really Recursing" (<https://mvanier.livejournal.com/2897.html>). Archived (<https://web.archive.org/web/20110822202348/http://mvanier.livejournal.com/2897.html>) from the original on 2011-08-22. "More generally, Y gives us a way to get recursion in a programming language that supports first-class functions but that doesn't have recursion built in to it."
23. The If Works Deriving the Y combinator (<https://blog.jcoglan.com/2008/01/10/deriving-the-y-combinator/>), January 10, 2008
 - Werner Kluge, *Abstract computing machines: a lambda calculus perspective*, Springer, 2005, ISBN 3-540-21146-2, pp. 73–77
 - Mayer Goldberg, (2005) *On the Recursive Enumerability of Fixed-Point Combinators* (<http://www.brics.dk/RS/05/1/BRICS-RS-05-1.pdf>), BRICS Report RS-05-1, University of Aarhus
 - Matthias Felleisen, A Lecture on the Why of Y (<https://docs.google.com/viewer?url=https%3A%2F%2Fwww.ps.uni-saarland.de%2Fcourses%2Fsem-prog97%2Fmaterial%2FYWorks.ps&embedded=true&chrome=false&dov=1>).

External links

- Recursion Theory and Joy (<https://hypercubed.github.io/joy/html/j05cmp.html>), Manfred von Thun, (2002 or earlier)
 - The Lambda Calculus - notes by Don Blaheta, October 12, 2000 (<https://cs.brown.edu/courses/cs173/2002/Lectures/2002-10-28-lc.pdf>)
 - Y Combinator (<http://www.ece.uc.edu/~franco/C511/html/Scheme/ycomb.html>) Archived (<https://web.archive.org/web/20090323042654/http://www.ece.uc.edu/~franco/C511/html/Scheme/ycomb.html>) 2009-03-23 at the Wayback Machine
 - "A Use of the Y Combinator in Ruby" (<https://www.sbf5.com/~cduan/technical/ruby/ycombinator.shtml>)
 - "Functional programming in Ada" (<https://okasaki.blogspot.com/2008/07/functional-programming-inada.html>)
 - Rosetta code - Y combinator (https://rosettacode.org/wiki/Y_combinator)
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Fixed-point_combinator&oldid=1332916107"